Performing TLS Fingerprinting on Network Namespaces

Samyak Gedam (252IS032)

March 2026

Aim:

To analyze and compare TLS ClientHello fingerprints generated by different TLS clients(TLS v1.2 and v1.3) using packet captures and the Mercury fingerprinting framework.

Objective:

- Capture TLS handshake traffic in an isolated Linux network namespace environment.
- Generate TLS connections using different clients.
- Extract TLS fingerprints using Mercury.
- Compare fingerprints to observe differences in cipher suites and extensions.

Tools Used:

Linux Network Namespaces, tcpdump, OpenSSL, curl, Cisco Mercury.

Network Namespaces Topology:

The experiment was conducted in an isolated virtual network created using Linux network namespaces.

- red: TLS client namespace
- router: packet capture namespace
- blue: TLS server namespace

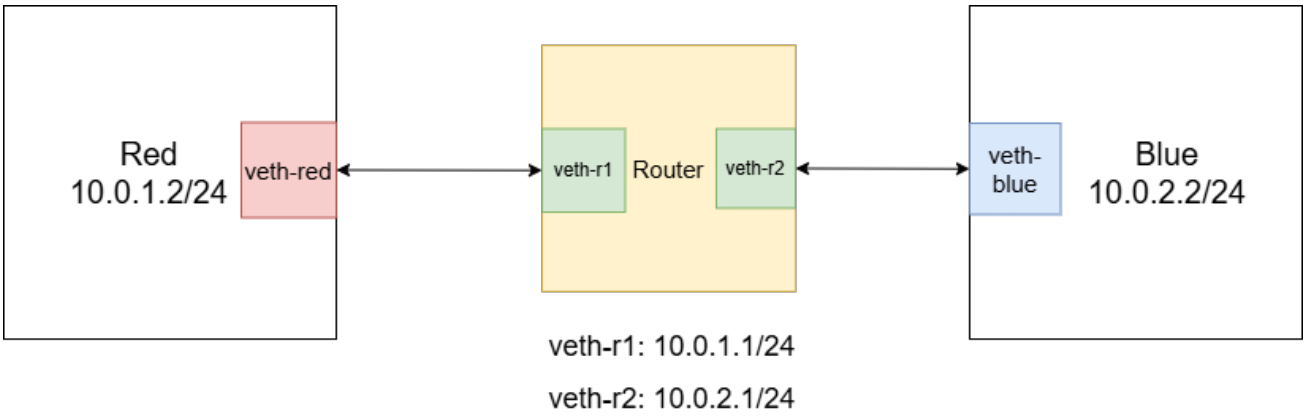

Figure 1: Network Topology

Packet Captures:

The following PCAP files were generated.

- curl_tls12.pcap
- curl_tls13.pcap
- python_ssl.pcap
- openssl.pcap

Capturing TLS Fingerprint:

Mercury was used to extract fingerprints.

```
./src/mercury -r ../pcaps/<name>.pcap
```

Example output:

`tls/(0303)(130213031301...)`

Fingerprint format:

`tls/(TLS_VERSION)(CIPHER_SUITES)(EXTENSIONS)`

Screenshots:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   bash - TLS_Fingerprint

• samyak@Samyak:~/projects/TLS_Fingerprintmain$ for f in pcaps/*.pcap; do
 > echo "===== $f ====="
 > ./mercury/src/mercury -r "$f" | jq 'select(.fingerprints.tls)'
 > echo
 > done
 ===== pcaps/curl_tls12.pcap =====
 {
   "fingerprints": {
     "tls": "tls/(0303)(c02cc030009fcca9cca8ccaac02bc02f009ec024c028006bc023c0270067c00ac0140039c009c0130033009d009c003d003c0035002f00ff)((000b00
0403000102)(000a000c000a001d0017001e00190018)(3374)(0010000e000c026832086874747 02f312e31)(0016)(0017)(000d002a002804030503060308070808080908 0a08
0b0804080508060401050106010303030103020402050206 02))"
   },
   "tls": {
     "client": {}
   },
   "src_ip": "10.0.1.2",
   "dst_ip": "10.0.2.2",
   "protocol": 6,
   "src_port": 53436,
   "dst_port": 4433,
   "event_start": 1772635639.956197
 }
```

Figure 2: Client - curl TLS1.2

```
===== pcaps/curl_tls13.pcap =====
{
  "fingerprints": {
    "tls": "tls/(0303)(13021303130100ff)((000b000403000102)(000a00160014001d0017001e0019001801000101010201030104)(3374)(0010000e000c026832086874
74702f312e31)(0016)(0017)(0031)(000d001e001c0403050306030807080808090 80a080b0804080508060401050106 01)(002b0003020304)(002d00020101)(0033))"
  },
  "tls": {
    "client": {}
  },
  "src_ip": "10.0.1.2",
  "dst_ip": "10.0.2.2",
  "protocol": 6,
  "src_port": 42532,
  "dst_port": 4433,
  "event_start": 1772635694.716018
}
```

Figure 3: Client - curl TLS1.3

```
===== pcaps/openssl.pcap =====
{
  "fingerprints": {
    "tls": "tls/(0303)(130213031301c02cc030009fcca9cca8ccaac02bc02f009ec024c028006bc023c0270067c00ac0140039c009c0130033009d009c003d003c0035002f0
0ff)((000b000403000102)(000a00160014001d0017001e0019001801000101010201030104)(0023)(0016)(0017)(000d002a002804030503060308070808080908 0a080b0804
0805080604010501060103030301030204020502060 2)(002b00050403040303)(002d00020101)(0033))"
  },
  "tls": {
    "client": {}
  },
  "src_ip": "10.0.1.2",
  "dst_ip": "10.0.2.2",
  "protocol": 6,
  "src_port": 34000,
  "dst_port": 4433,
  "event_start": 1772635564.716025
}
```

Figure 4: Client - OpenSSL

```
===== pcaps/python_ssl.pcap =====
{
  "fingerprints": {
    "tls": "tls/(0303)(130213031301c02cc030c02bc02fcca9cca8c024c028c023c027009f009e006b006700ff)((000b000403000102)(000a00160014001d0017001e0019
0018010001010102010301 04)(0023)(0016)(0017)(000d002a002804030503060308070808080908 0a080b0804080508060401050106010303030103020402050206 02)(002b00
050403040303)(002d00020101)(0033)(0015))"
  },
  "tls": {
    "client": {}
  },
  "src_ip": "10.0.1.2",
  "dst_ip": "10.0.2.2",
  "protocol": 6,
  "src_port": 46228,
  "dst_port": 4433,
  "event_start": 1772635734.752701
}
```

Figure 5: Client - Python SSL

results > { } captured_fingerprints.json > ...


```
{
  "fingerprints": {
    "tls": "tls/(0303)
    (c02cc030009fcca9cca8ccaac02bc02f009ec024c028006bc023c0270067c00ac0140
    039c009c0130033009d009c003d003c0035002f00ff)((000b000403000102)
    (000a000c000a001d0017001e00190018)(3374)
    (0010000e000c02683208687474702f312e31)(0016)(0017)
    (000d002a002804030503060308070808080908 0a080b08040805080604010501060 10
    303030103020402050206 02))"
  },
  "tls": {
    "client": {}
  },
  "src_ip": "10.0.1.2",
  "dst_ip": "10.0.2.2",
  "protocol": 6,
  "src_port": 53436,
  "dst_port": 4433,
  "event_start": 1772635639.956197
}
{
  "fingerprints": {
    "tls": "tls/(0303)(13021303130100ff)((000b000403000102)
    (000a00160014001d0017001e001900180100010101020103 0104)(3374)
    (0010000e000c02683208687474702f312e31)(0016)(0017)(0031)
    (000d001e001c040305030603080708080809080a080b08040805080604010501060 1)
    (002b0003020304)(002d00020101)(0033))"
```

results > { } captured_fingerprints.json > ...


```
18    },
19    "tls": {
20      "client": {}
21    },
22    "src_ip": "10.0.1.2",
23    "dst_ip": "10.0.2.2",
24    "protocol": 6,
25    "src_port": 42532,
26    "dst_port": 4433,
27    "event_start": 1772635694.716018
28  }
29  {
30    "fingerprints": {
31      "tls": "tls/(0303)
      (130213031301c02cc030009fcca9cca8ccaac02bc02f009ec024c028006bc023c0270
      067c00ac0140039c009c0130033009d009c003d003c0035002f00ff)
      ((000b000403000102)
      (000a00160014001d0017001e00190018010001010101020103010 4)(0023)(0016)
      (0017)
      (000d002a002804030503060308070808080908 0a080b08040805080604010501060 10
      303030103020402050206 02)(002b00050403040303)(002d00020101)(0033))"
32    },
33    "tls": {
34      "client": {}
35    },
36    "src_ip": "10.0.1.2",
37    "dst_ip": "10.0.2.2",
```

results > { } captured_fingerprints.json > ...


```
38    "protocol": 6,
39    "src_port": 34000,
40    "dst_port": 4433,
41    "event_start": 1772635564.716025
42  }
43  {
44    "fingerprints": {
45      "tls": "tls/(0303)
        (130213031301c02cc030c02bc02fcca9cca8c024c028c023c027009f009e006b00670
        0ff)((000b000403000102)
        (000a00160014001d0017001e001900180100010101020103010 4)(0023)(0016)
        (0017)
        (000d002a002804030503060308070808080908 0a080b08040805080604010501060 10
        303030103020402050206 02)(002b00050403040303)(002d00020101)(0033)
        (0015))"
46    },
47    "tls": {
48      "client": {}
49    },
50    "src_ip": "10.0.1.2",
51    "dst_ip": "10.0.2.2",
52    "protocol": 6,
53    "src_port": 46228,
54    "dst_port": 4433,
55    "event_start": 1772635734.752701
56  }
57
```

Results

The extracted fingerprints from different TLS clients were compared.

Client	TLS Version	Observation
curl TLS1.2	TLS 1.2	Large cipher suite list
curl TLS1.3	TLS 1.3	Modern AEAD cipher suites
Python SSL	TLS 1.3	Different extension ordering
OpenSSL	TLS 1.2	Minimal handshake structure

Table 1: Comparison of TLS fingerprints

Discussion

The results show that different TLS implementations produce distinct fingerprints even when connecting to the same server. The differences arise from:

- Cipher suite ordering
- Supported TLS versions
- TLS extensions
- Client library implementation

Feature	TLS 1.2	TLS 1.3
Cipher Suites	Large and diverse list	Short modern list
Typical Ciphers	c02c, c030, 009f, cca9	1301, 1302, 1303
New Extensions	Not present	supported_versions, key_share
Handshake Structure	Multiple negotiation steps	Simplified and faster
Fingerprint Pattern	Longer and more variable	Shorter but distinct

Table 2: Comparison of TLS 1.2 and TLS 1.3 fingerprints

Other TLS Fingerprinting Tools

I identified several other tools besides Mercury. **JA3** is one of the most widely used methods and generates fingerprints by hashing fields from the TLS ClientHello message.
Some other Tools such as **Wireshark, Zeek, and Suricata** also support JA3 fingerprint extraction from packet captures.
But in comparison, **Mercury** provides more detailed fingerprints without hashing the handshake structure, which helped me for better analysis of TLS client behavior.

Conclusion

Demonstrated how TLS fingerprinting can identify different TLS clients using handshake metadata. By capturing traffic and analyzing fingerprints with the Mercury framework, it was observed that each client produces a unique fingerprint due to variations in cipher suites and extensions.